

Fully Homomorphic Encryption (FHE)

Part I: Intuition, Mathematical Essentials, and Historical Homomorphism

Ksawery Moźdzynski

Agenda for Today



1. Introduction to FHE

Intuition, the Avalanche Effect, and Cloud Computing



2. Mathematical Essentials

Modular Arithmetic, Cyclic Groups, and DLP



3. Introduction to the ElGamal Scheme

The Magic of Multiplicative Homomorphism



4. Practical Exercises

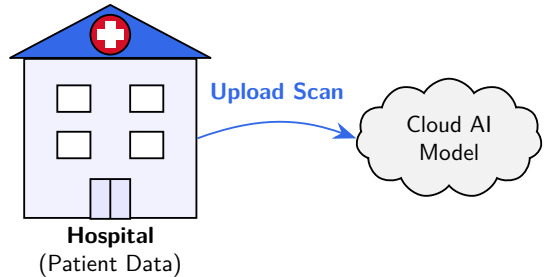
Python Implementation and Generating Chaos

The Goal: AI in Healthcare

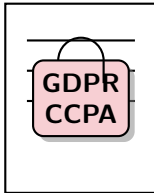
Scenario:

- We want to build an innovative cloud service for healthcare.
- A hospital uploads a patient's MRI scan.
- The cloud runs advanced AI models (neural network inferences) for early disease detection.

Value: Fast, scalable, and highly accurate diagnosis without local hardware constraints.



The Barrier: Strict Privacy Regulations



The Problem:

- Clients cannot share sensitive healthcare records with untrusted cloud providers.
- Modern legislation (EU GDPR, California's CCPA) mandates strict data protection.
- **Conclusion:** Data **MUST** be encrypted before leaving the hospital. Plaintext transfer is illegal.

The Cloud Privacy Paradox

The Dilemma: If data must be encrypted (e.g., using standard AES), the cloud cannot access it to perform AI analysis.



Traditional cryptography is designed **only for data confidentiality**, not for computation!

Traditional Cryptography: Data Confidentiality

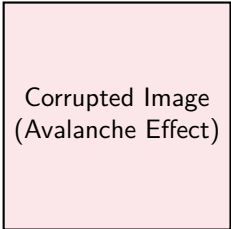


Data Confidentiality

The fundamental goal of hiding data from unauthorized parties. Typically achieved using symmetric encryption algorithms, such as the **AES (Advanced Encryption Standard)**.

Naive Approach: Computing on AES Ciphertexts?

What if the cloud server simply tries to apply a mathematical filter directly to the encrypted AES bytes?



Corrupted Image
(Avalanche Effect)

The Avalanche Effect:

- Modifying even a single byte of an AES ciphertext results in a destroyed plaintext.
- Any naive mathematical operation on ciphertext bytes irreversibly destroys the data.

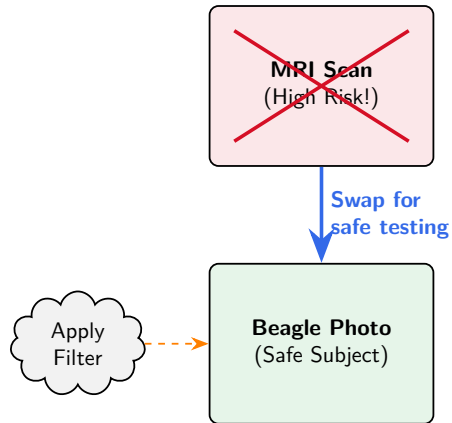
Testing the Naive Approach: A Safe Experiment

The Naive Idea:

- What if the cloud server simply ignores the encryption and applies a mathematical filter directly to the AES ciphertext bytes?
- Before we risk irreversibly destroying a patient's sensitive MRI scan, we must test this hypothesis safely.

Our Test Subject:

- Instead of a medical record, let's use a harmless, standard image: **a photo of a Beagle dog**.
- We will ask the cloud to apply some changes to the encrypted dog photo. Let's see what happens...



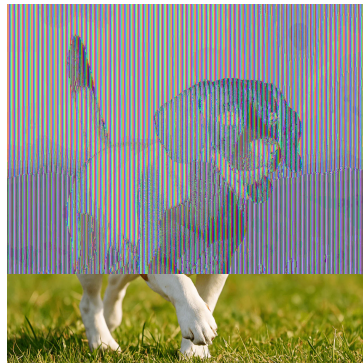
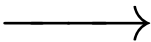
The Fragility of Traditional Ciphers (AES PCBC Mode)

The Avalanche Effect: Modifying the ciphertext destroys the decrypted data.



dog.bmp
(Original)

1 byte modified in ciphertext
*Corrupts one AES block (16 bytes)
propagates error to subsequent blocks
due to PCBC mode*

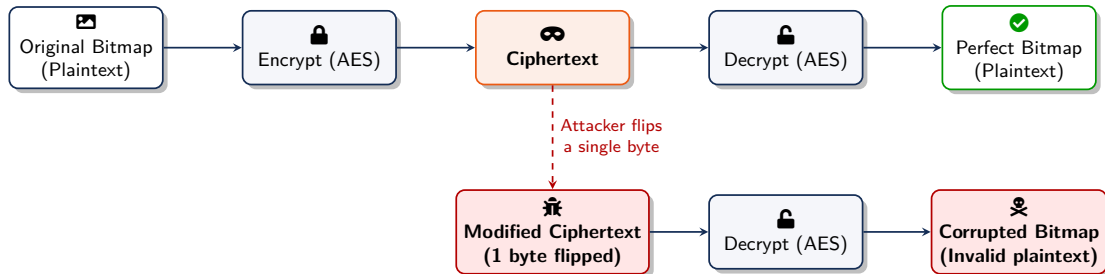


dog_corrupted.bmp
(Decrypted)

The Fragility of Traditional Ciphers

! The Avalanche Effect in AES

In traditional ciphers designed for confidentiality, **modifying the ciphertext destroys the underlying data.**



The Goal: Computing on Encrypted Data

For simplicity, let's say the modification is a basic filter:
making the center of the photo grayscale.

The Problem

How do we build such a service?

As we just saw, if we try to modify **AES ciphertexts**, we will destroy the image!



Desired Output

Naive Approach: Modifying the Ciphertext

The Naive Attempt:

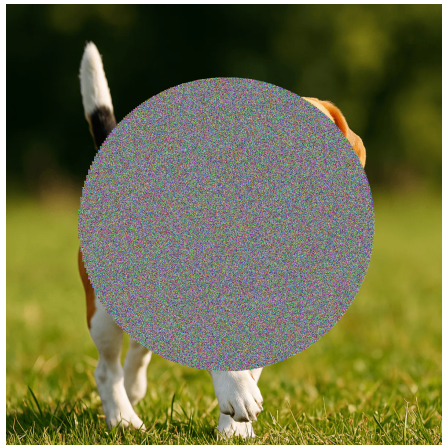
What if the cloud server simply applies the grayscale filter directly to the *encrypted* pixels?

The Result:

Because of the **Avalanche Effect**, any mathematical operation on the ciphertext bytes irreversibly destroys the data. Even in ECB mode (Electronic Codebook), used here to limit error propagation.

Decryption Failure

Instead of a grayscale center, the user decrypts a completely corrupted hole of random noise!



Decrypted Result

The Solution: Fully Homomorphic Encryption (FHE)

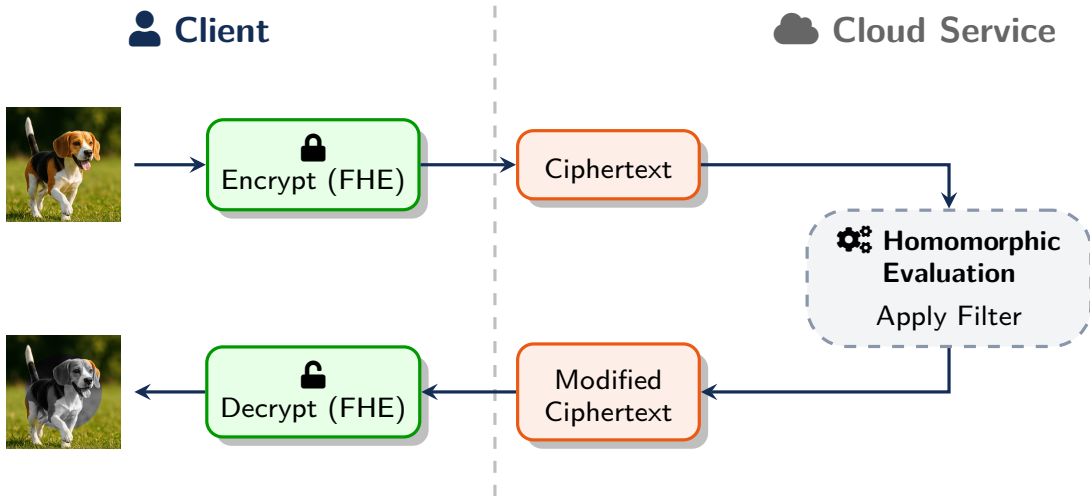
The Challenge Resolved: What if we *want* to modify the ciphertext in a meaningful way to perform computations without decrypting it?

What is FHE?

Fully Homomorphic Encryption (FHE) is a cryptographic scheme that enables computations to be performed directly on encrypted data, as if the data were in plaintext.

Unlike traditional ciphers, FHE allows the cloud server to apply our grayscale filter directly to the encrypted bytes without ever seeing the original photo.

FHE in Action: Secure Cloud Filtering



Key Applications of FHE

Because FHE allows computations on hidden data, it unlocks entirely new paradigms for data security:

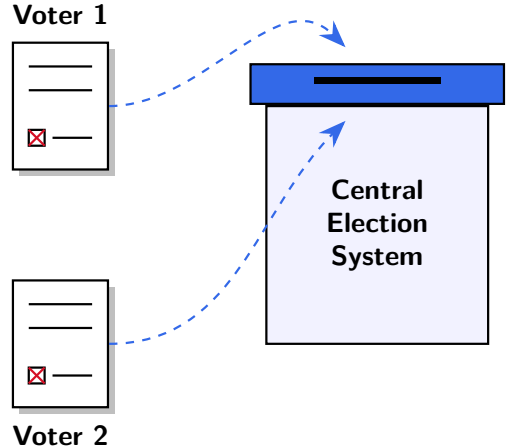
-  **Privacy-Preserving Cloud Computing:** Secure outsourcing of data analytics and encrypted database queries to untrusted cloud servers.
-  **Secure Machine Learning:** Allowing a server to process a client's encrypted data through an ML model without exposing the input features or the inference results.
-  **Confidential Blockchain Services:** Ensuring that sensitive data in smart contracts remains encrypted and confidential while maintaining execution transparency.

Another Application: Secure Digital Voting

Scenario:

- We want to build a fully secure, remote digital voting system for national elections.
- Every citizen submits their ballot electronically.

Value: Maximum convenience and high voter turnout, but it requires absolute mathematical trust in the election integrity.



The Voting Dilemma: Privacy vs. Verifiability

Encrypted Ballot

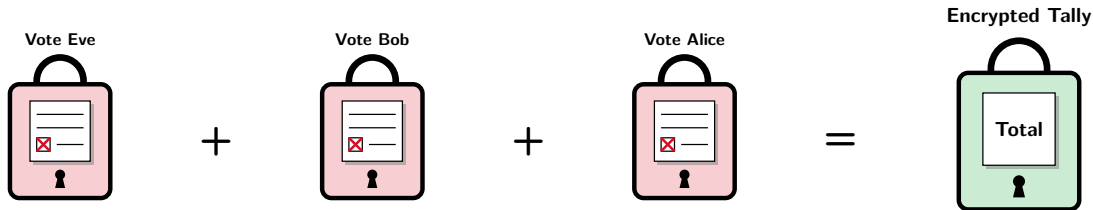


The Problem:

- To ensure voter privacy, ballots must be strictly encrypted before leaving the device.
- However, to tally the results, a standard cryptographic system (like AES) requires the central server to **decrypt** the votes first.
- **Privacy Compromised:** The system admin or a hacker could see exactly how every individual voted!

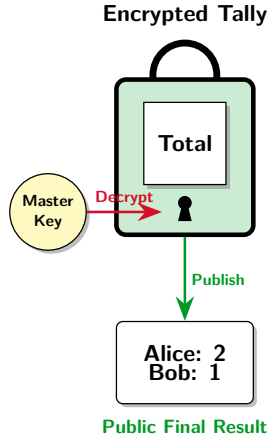
The FHE Solution: Homomorphic Addition

Fully Homomorphic Encryption allows the server to mathematically tally the results while the votes remain completely encrypted!



Computed completely blind

Absolute Election Integrity



How FHE ensures absolute trust:

- The system mathematically guarantees that the final count is 100% correct.
- **Privacy preserved:** No individual vote is ever decrypted during the process.
- Only the **final, aggregated result** is decrypted by the election committee's master key.

A Stepping Stone: Historical Homomorphism

Before we dive into modern Fully Homomorphic Encryption (FHE) algorithms, it is worth looking at the simplest example of a classic cipher that already allows **partial modification** of encrypted data (specifically, through multiplication).

The ElGamal Scheme

We will explore the ElGamal encryption scheme as our first historical example of *multiplicative homomorphism*.

But first...

To understand exactly how this works under the hood, let's quickly recall some mathematical essentials!

Mathematical Essentials: Modular Arithmetic

The Foundation of Cryptography Rings and Fields

Modulo (mod) is the operation of computing the remainder when one number is divided by another.

Key Concept: Congruence ($\equiv$)

- We say that two numbers are congruent modulo q if they have the same remainder when divided by that modulus q .

Simple Examples

$$12 = 2 \cdot 5 + 2 \quad (\text{The remainder of dividing 12 by 5 is } 2)$$

$$7 = 1 \cdot 5 + 2 \quad (\text{The remainder of dividing 7 by 5 is } 2)$$

- $12 \equiv 7 \pmod{5}$ (both 12 and 7 leave a remainder of 2 when divided by 5)

Mathematical Essentials: Modular Multiplication

How to Multiply

Multiply the numbers as usual in traditional math, then simply take the remainder (modulo).

Standard Example (Modulo 7)

Let's compute $5 \cdot 16 \pmod{7}$:

- First, multiply normally: $5 \cdot 16 = 80$.
- Then, divide by 7 to find the remainder.
Since $80 = 11 \cdot 7 + 3$, the remainder is **3**.

Result: **$5 \cdot 16 \equiv 3 \pmod{7}$**

Mathematical Essentials: Modular Exponentiation

What is it?

Exponentiation (g^x) is simply repeated modular multiplication.

Example (Modulo 7)

Let's compute $3^4 \pmod{7}$:

- First, expand the exponent: $3^4 = 3 \cdot 3 \cdot 3 \cdot 3 = 81$.
- Then, find the remainder: Since $81 = 11 \cdot 7 + 4$, the remainder is **4**.

Result: $3^4 \equiv 4 \pmod{7}$

Mathematical Essentials: The Power of Wrapping Around

Why is it crucial for Cryptography?

Why do we use modular math instead of regular math to encrypt data?

- **Bounded Growth:** In modular arithmetic, values never grow to infinity.
- **The Clock Effect:** They continuously “wrap around” the modulus, much like the hands on a clock.
- **Data Masking:** This wrapping behavior completely destroys obvious mathematical patterns (like size or linearity). It is the core mechanism used to **mask data**!

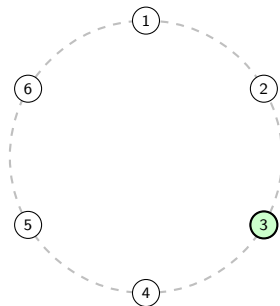
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv 3 \pmod{7}$



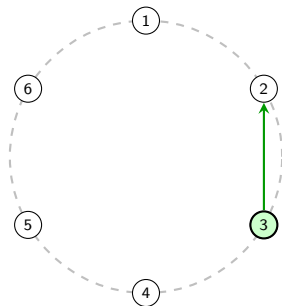
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv 3 \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv 3 \cdot 3 \equiv 9 \equiv 2 \pmod{7}$



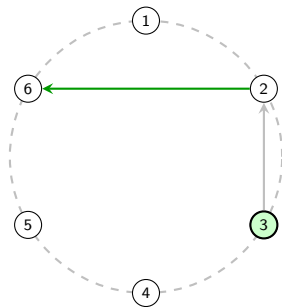
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv 3 \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv 3 \cdot 3 \equiv 9 \equiv 2 \pmod{7}$
- $3^3 = 3^2 \cdot 3 \equiv 2 \cdot 3 \equiv 6 \equiv 6 \pmod{7}$



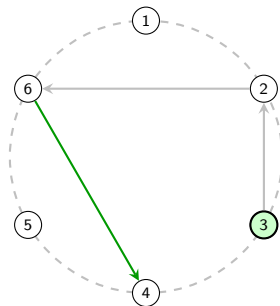
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv \mathbf{3} \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv \mathbf{3} \cdot 3 \equiv 9 \equiv \mathbf{2} \pmod{7}$
- $3^3 = 3^2 \cdot 3 \equiv \mathbf{2} \cdot 3 \equiv 6 \equiv \mathbf{6} \pmod{7}$
- $3^4 = 3^3 \cdot 3 \equiv \mathbf{6} \cdot 3 \equiv 18 \equiv \mathbf{4} \pmod{7}$



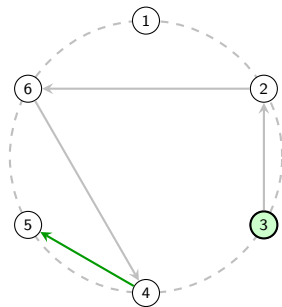
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv \mathbf{3} \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv \mathbf{3} \cdot 3 \equiv 9 \equiv \mathbf{2} \pmod{7}$
- $3^3 = 3^2 \cdot 3 \equiv \mathbf{2} \cdot 3 \equiv 6 \equiv \mathbf{6} \pmod{7}$
- $3^4 = 3^3 \cdot 3 \equiv \mathbf{6} \cdot 3 \equiv 18 \equiv \mathbf{4} \pmod{7}$
- $3^5 = 3^4 \cdot 3 \equiv \mathbf{4} \cdot 3 \equiv 12 \equiv \mathbf{5} \pmod{7}$



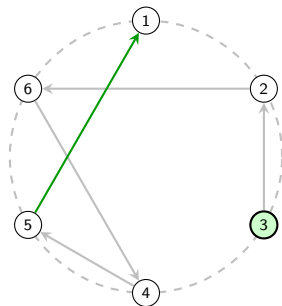
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv \mathbf{3} \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv \mathbf{3} \cdot 3 \equiv 9 \equiv \mathbf{2} \pmod{7}$
- $3^3 = 3^2 \cdot 3 \equiv \mathbf{2} \cdot 3 \equiv 6 \equiv \mathbf{6} \pmod{7}$
- $3^4 = 3^3 \cdot 3 \equiv \mathbf{6} \cdot 3 \equiv 18 \equiv \mathbf{4} \pmod{7}$
- $3^5 = 3^4 \cdot 3 \equiv \mathbf{4} \cdot 3 \equiv 12 \equiv \mathbf{5} \pmod{7}$
- $3^6 = 3^5 \cdot 3 \equiv \mathbf{5} \cdot 3 \equiv 15 \equiv \mathbf{1} \pmod{7}$



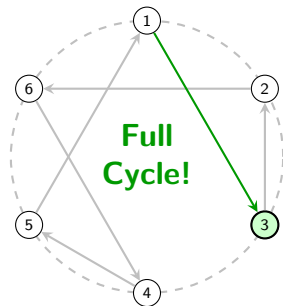
Visualizing Generators: The Cryptographic Clock

Repeated Multiplication (Modulo 7)

Let's see what happens when we repeatedly multiply by number 3.

Let's call this number our **generator**.

- $3^1 \equiv \mathbf{3} \pmod{7}$
- $3^2 = 3^1 \cdot 3 \equiv \mathbf{3} \cdot 3 \equiv 9 \equiv \mathbf{2} \pmod{7}$
- $3^3 = 3^2 \cdot 3 \equiv \mathbf{2} \cdot 3 \equiv 6 \equiv \mathbf{6} \pmod{7}$
- $3^4 = 3^3 \cdot 3 \equiv \mathbf{6} \cdot 3 \equiv 18 \equiv \mathbf{4} \pmod{7}$
- $3^5 = 3^4 \cdot 3 \equiv \mathbf{4} \cdot 3 \equiv 12 \equiv \mathbf{5} \pmod{7}$
- $3^6 = 3^5 \cdot 3 \equiv \mathbf{5} \cdot 3 \equiv 15 \equiv \mathbf{1} \pmod{7}$



The Quest for a "Good" Generator

The Core Engine of Cryptography

Repeated modular multiplication and exponentiation are computed **constantly** in modern cryptography (e.g., the ElGamal scheme).

What makes a generator "Good"?

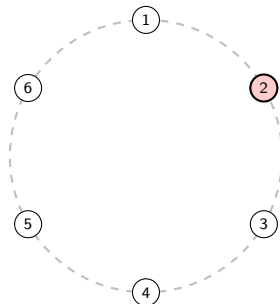
- We want to choose a base number (generator g) that visits **as many points as possible** before returning to 1.
- Ideally, it should visit *every single valid number* in our modulo group.

Visualizing Generators: A "Bad" Generator

Repeated Multiplication (Modulo 7)

Now, let's see what happens if we choose number 2 as our generator instead.

- $2^1 \equiv 2 \pmod{7}$

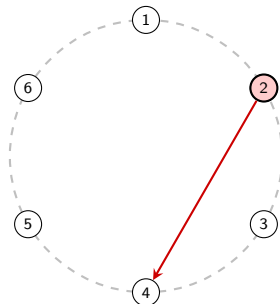


Visualizing Generators: A "Bad" Generator

Repeated Multiplication (Modulo 7)

Now, let's see what happens if we choose number 2 as our generator instead.

- $2^1 \equiv 2 \pmod{7}$
- $2^2 = 2^1 \cdot 2 \equiv 2 \cdot 2 \equiv 4 \pmod{7}$

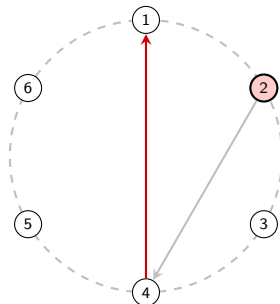


Visualizing Generators: A "Bad" Generator

Repeated Multiplication (Modulo 7)

Now, let's see what happens if we choose number 2 as our generator instead.

- $2^1 \equiv \mathbf{2} \pmod{7}$
- $2^2 = 2^1 \cdot 2 \equiv \mathbf{2} \cdot 2 \equiv \mathbf{4} \pmod{7}$
- $2^3 = 2^2 \cdot 2 \equiv \mathbf{4} \cdot 2 \equiv 8 \equiv \mathbf{1} \pmod{7}$

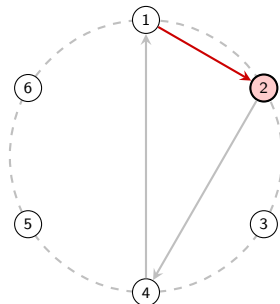


Visualizing Generators: A "Bad" Generator

Repeated Multiplication (Modulo 7)

Now, let's see what happens if we choose number 2 as our generator instead.

- $2^1 \equiv \mathbf{2} \pmod{7}$
- $2^2 = 2^1 \cdot 2 \equiv \mathbf{2} \cdot 2 \equiv \mathbf{4} \pmod{7}$
- $2^3 = 2^2 \cdot 2 \equiv \mathbf{4} \cdot 2 \equiv 8 \equiv \mathbf{1} \pmod{7}$
- $2^4 = 2^3 \cdot 2 \equiv \mathbf{1} \cdot 2 \equiv \mathbf{2} \pmod{7}$

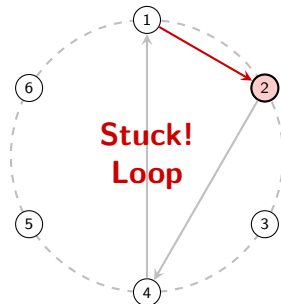


Visualizing Generators: A "Bad" Generator

Repeated Multiplication (Modulo 7)

Now, let's see what happens if we choose number 2 as our generator instead.

- $2^1 \equiv \mathbf{2} \pmod{7}$
- $2^2 = 2^1 \cdot 2 \equiv \mathbf{2} \cdot 2 \equiv \mathbf{4} \pmod{7}$
- $2^3 = 2^2 \cdot 2 \equiv \mathbf{4} \cdot 2 \equiv 8 \equiv \mathbf{1} \pmod{7}$
- $2^4 = 2^3 \cdot 2 \equiv \mathbf{1} \cdot 2 \equiv \mathbf{2} \pmod{7}$



The Core of Security: Discrete Logarithm Problem

The Trapdoor Function (One-Way Street)

In modular arithmetic, the equation $y \equiv g^x \pmod{p}$ is easy to compute in one direction, but practically impossible to reverse.

*The upcoming **ElGamal cipher** is built entirely on this mathematical foundation!*



Be Careful: The Illusion of Security

Math is only as strong as its parameters!

Cryptography is secure **only if** we choose:

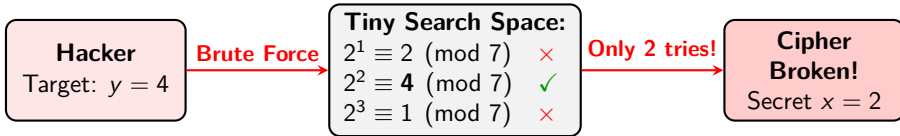
- Sufficiently large numbers (modulus).
- Proper cryptographic parameters.
- A "good" generator (Primitive Root).

What happens if we ignore this?

- If we pick a bad generator the cycle is incredibly small (only 3 elements).
- The Discrete Logarithm Problem (DLP) completely loses its "one-way" property.
- An attacker can easily guess the secret exponent x using simple **trial and error** (brute force)!

The Danger of a "Bad" Generator: Brute Force Attack

Scenario: A hacker intercepts ciphertext $y = 4$ and knows public parameters $g = 2, p = 7$. They want to find the secret x in $2^x \equiv 4 \pmod{7}$.



A small subgroup completely destroys the "one-way" property of the DLP!

Asymmetric Cryptography: Visual Concept

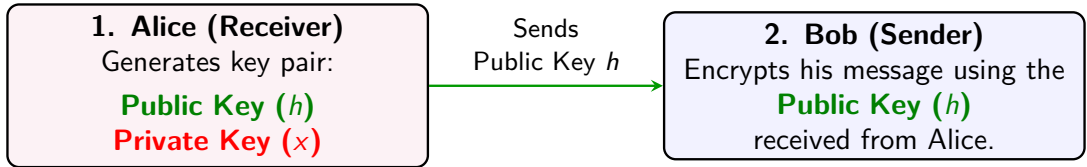
1. Alice (Receiver)

Generates key pair:

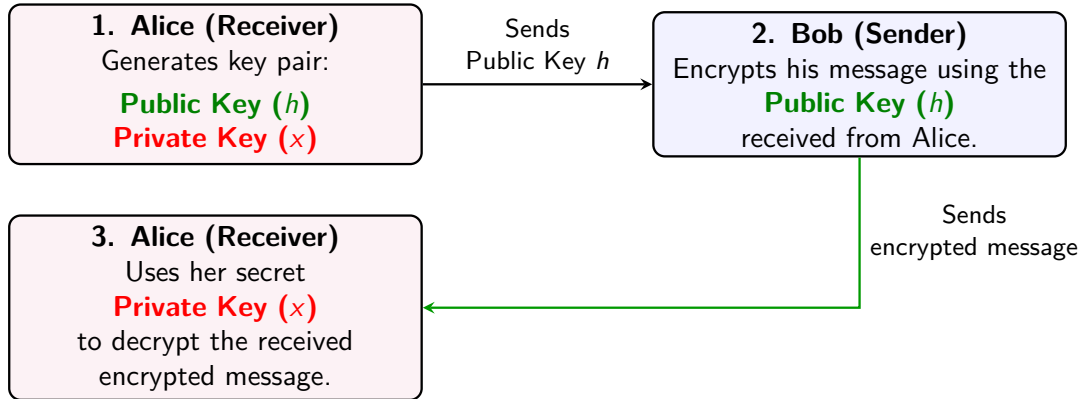
Public Key (h)

Private Key (x)

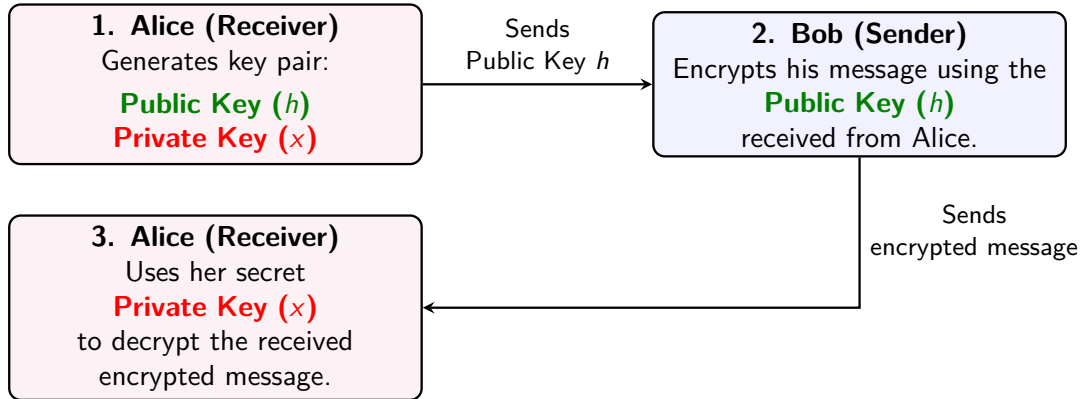
Asymmetric Cryptography: Visual Concept



Asymmetric Cryptography: Visual Concept



Asymmetric Cryptography: Visual Concept



Historical Homomorphism: The ElGamal Scheme

1. Key Generation

- Choose a large prime number p and a generator g .
- Choose a random private key x .
- Compute public key $h \equiv g^x \pmod{p}$.

2. Encryption

Message m . Choose random y .

Compute ciphertext $C = (c_1, c_2)$:

- $c_1 \equiv g^y \pmod{p}$
- $c_2 \equiv m \cdot h^y \pmod{p}$

3. Decryption

Given $C = (c_1, c_2)$ and private key x :

Recover message m :

- $s \equiv c_1^x \pmod{p}$
- $m \equiv c_2 \cdot s^{-1} \pmod{p}$

ElGamal Toy Example (1/3): Key Generation

Setup: We use our "good" generator $g = 3$ in $\mathbb{Z}_7^*$ (modulus $p = 7$).

Step 1: Receiver generates keys

- **Private Key (x):** Choose a random number, e.g., $x = 2$.
- **Public Key (h):** Compute $h \equiv g^x \pmod{p} \equiv 3^2 \pmod{7} = 9 \equiv 2$

Result: The Receiver publicly shares $(p = 7, g = 3, h = 2)$ and keeps $x = 2$ secret.

ElGamal Toy Example (2/3): Encryption

Scenario: Sender wants to encrypt the message $m = 5$ with $(p = 7, g = 3, h = 2)$.

Step 2: Sender encrypts the message

- **Random Element (y):** Choose a temporary random number, e.g., $y = 4$.
- **Compute c_1 (Masked Key):**

$$\begin{aligned}c_1 &\equiv g^y \pmod{p} \\c_1 &\equiv 3^4 \equiv 81 \equiv 4 \pmod{7}\end{aligned}$$

- **Compute c_2 (Masked Message):**

$$\begin{aligned}c_2 &\equiv m \cdot h^y \pmod{p} \\c_2 &\equiv 5 \cdot 2^4 \equiv 5 \cdot 16 \equiv 80 \equiv 3 \pmod{7}\end{aligned}$$

Result: Sender outputs Ciphertext $C = (c_1, c_2) = (4, 3)$.

ElGamal Toy Example (3/3): Decryption

Scenario: Receiver gets Ciphertext $C = (4, 3)$ and uses Private Key $x = 2$ with $(p = 7, g = 3, h = 2)$.

Step 3: Receiver decrypts the ciphertext

- Recover Shared Secret (s):

$$s \equiv c_1^x \pmod{p}$$
$$s \equiv 4^2 \equiv 16 \equiv 2 \pmod{7}$$

- Recover Message (m):

$$s^{-1} \equiv 2^{-1} \equiv 4 \pmod{7}$$
$$m \equiv c_2 \cdot s^{-1} \pmod{p}$$
$$m \equiv 3 \cdot 4 = 12 \equiv 5 \pmod{7}$$

The Magic of Multiplicative Homomorphism

Why ElGamal in an FHE course?

Because it natively possesses a *multiplicative homomorphic* property!

This means that if we take two encrypted messages and simply multiply them together, the result is a valid ciphertext of their product. We do not need the decryption key to do this!

The Core Homomorphic Property

$$\text{Enc}(m_A) \cdot \text{Enc}(m_B) = \text{Enc}(m_A \cdot m_B)$$

But is this actually true? Let's prove it on the next slide!

Proving the Homomorphism

Multiplying Two Ciphertexts

Suppose we have two ciphertexts encrypting m_A and m_B :

- $C_A = (c_{1A}, c_{2A}) = (g^{y_A}, m_A \cdot h^{y_A})$
- $C_B = (c_{1B}, c_{2B}) = (g^{y_B}, m_B \cdot h^{y_B})$

If we multiply their components together:

- $c_{1A} \cdot c_{1B} = g^{y_A} \cdot g^{y_B} = \mathbf{g^{y_A+y_B}}$
- $c_{2A} \cdot c_{2B} = (m_A h^{y_A}) \cdot (m_B h^{y_B}) = (\mathbf{m_A \cdot m_B}) \cdot \mathbf{h^{y_A+y_B}}$

The resulting ciphertext $(c_{1A} \cdot c_{1B}, c_{2A} \cdot c_{2B})$ has the exact same mathematical structure as a fresh encryption of $(m_A \cdot m_B)$ using the random value $y_A + y_B$.

Practical Exercises

- **Task 1: Implement ElGamal Encryption Scheme in Python.** Implement key generation, encryption, and decryption.
- **Task 2: Design Homomorphic Encryption.** Extend the basic ElGamal scheme to demonstrate its multiplicative homomorphism.
- **Task 3: Implement Homomorphic Multiplication Operation.** Write a function that takes two ciphertexts as input and outputs the ciphertext of their product.

Bonus Tasks:

- *The Art of Generating Chaos:* Computationally find a "good" generator (a primitive root) for the group $\mathbb{Z}_p$, and intentionally select a "bad" generator to demonstrate how it fails to properly mask data.
- *Unit Testing:* Test the implementation using pytest.